Contents

Update an installed App to a newer version: `updateApp`

1. Problem
2. Use cases
3. Affected domains
4. Approach options
5. Recommended option
6. Flow and failure rules
7. Store and command shape
8. Test plan
9. Design review
10. Open questions

Update an installed App to a newer version: `updateApp`

Author: Steve · IDEA Lead Bot **Date:** 2026-10-28 **Status:** Draft. Design review done for Console, Engine, App and Ops. On 2026-10-28 Koen decided that v1 must also update from a newer bundle on a disk docked in **another** Engine on the school network ("Yes, add the network copy to the first version"). That adds the Fetch phase (§6). Kid, Atlas and Pixel have reviewed Fetch; Axle's review (transport choice) is pending (§10).

Covers agent-engine-dev#230 (approved to build by Koen, 2026-10-28) and the `upgradeApp` half of idea#76. Touches idea#74 (Upgrade Disk) and idea#77 (Engine self-upgrade). Code read at Engine main `2583608`.

1. Problem

An App instance is frozen at the version it was installed with. Once Kolibri 0.15.5 runs on a school's App Disk, there is no supported way to move that instance to a fixed or newer App version. The only options are:

- **Hand-edit** `/instances/<id>/compose.yaml` over SSH. This is what happened on the kit Engines on 2026-10-17 (the Kolibri Morango ID fix, app-kolibri#8), as a one-time exception.
- **Install a fresh instance** from the new version. This leaves the teacher's data in the old instance, and a duplicate name next to it (there's no remove command yet, idea#347).

Neither works in a school with no internet and no IT staff. Meanwhile the App Dev Bot ships App fixes every week (`app-update` issues, idea#88). Today they can't reach an installed school.

What the code already has: `upgradeApp` is an `OperationKind` with recovery `'fail' // not yet implemented` (`Operations.ts:38`). `isMajorUpgrade` / `extractMajorVersion` exist in `App.ts:47-65` and are unit-tested, but nothing calls them. The bundle (`apps/<appId>/`), the instance folder (`instances/<id>/` with `compose.yaml` , `.env` , `data/`) and the saved service images (`services/*.tar`) all live on the App Disk. `backupApp` / `restoreApp` (Borg) and `ResourceLock` already exist.

2. Use cases

1. **Field staff bring a newer App Disk.** A Kolibri 0.15.6 App Disk is docked next to the school's own disk, which runs Kolibri 0.15.5 with a year of learner progress. They want the school's instance on 0.15.6, keeping all its data. *(The core case; offline.)*

2. **A bug fix for an App that's already installed.** Same as 1, but the fix is a compose/env change (as app-kolibri#8). The source can be a docked disk, or GitHub when the Pi happens to be online (dev pool, Tailscale-reachable kits).
3. **The teacher sees "Update available"** in the Console after a newer disk was docked, and taps one button. No terminal, no choices beyond "Update" or "Not now". *(This is #76's dock-time part, built on the same command.)*
4. **The update goes wrong.** The new version won't start, or starts but serves errors. The school must end up on the old version with its data unchanged, without anyone touching it.
5. **A major version jump** (Nextcloud 27 to 29, Kolibri 0.x to 1.x) needs the App's own data migration, which may be one-way.

Constraints from these: offline sources first; one tap; never lose data; failure is undone automatically; the Pi is slow and the disk can be pulled at any moment.

3. Affected domains

Domain	Change
Engine (Axle)	`updateApp` command and `upgradeApp` operation; pre-update snapshot; health gate; rollback; dock-time "update available" detection (#76).
Console (Pixel)	"Update available" badge from `updateAvailable`, shown only when the Engine has the `updateApp` capability; confirm, progress and result dialogs (§7.3).
App Disks / Kid	`x-app` fields (`healthCheck`, `healthTimeout`, `updatesFrom`, `migrationNote`, `snapshotExclude`) in app-kolibri, app-nextcloud, app-kiwix and app-milkwise; a harness test "update from previous release with real data"; measured first-start times after migrations on a pool Pi to set the defaults.
Ops (Atlas)	Reviewed: disk space, backups, reboot safety, fleet rollout (§9).

4. Approach options

4.1 Where the new version runs

A. In-place swap. Stop the instance, keep `data/` and `.env`, replace the bundle-derived files (`compose.yaml` and the other bundle files, `services/*.tar`), re-apply the Engine's compose adjustments (`x-app.version`, `instanceName`, port, files/usb mounts), start. - Pros: same instance ID, name, port, URL, backups and peer references; no extra disk space for data; matches #230's proposal. -

Cons: the old version must be kept somewhere for rollback; an App that migrates `data/` on first start changes it in place.

B. Side-by-side new instance. Install the new version as a second instance, copy `data/` into it, start it, then retire the old one. - Pros: the old instance stays untouched until the switch, so rollback is trivial. - Cons: doubles the data on disk (Kolibri content can be tens of GB on a Pi SSD); a new instance ID breaks backup history, bookmarks and the Console's identity; two instances with the same port/hostname collide (the issue behind #230); needs remove (idea#347) to finish.

4.2 Protecting the data

- **None.** Unacceptable for case 4 and 5.
- **Full Borg backup** before update (`backupApp`). Strong, but needs a docked Backup Disk and is slow on a Pi.
- **Local snapshot**: before the swap, move the old bundle files to `instances/<id>/.pre-update/<oldAppId>/` and copy the instance's data there: `data/` **and its Docker named volumes** (for example Nextcloud's database volume), minus anything the App lists in `x-app.snapshotExclude` (for example Kolibri's `content/`, which is re-importable and large). Fast to restore, but costs disk space. The free-space check measures exactly this set plus a margin; without room the update is refused (`NO_SPACE`).

4.3 Deciding the update worked

The Engine already sees "containers running". That isn't enough: Kolibri can run and still return 500 after a failed migration. The new version passes only if, within `x-app.healthTimeout` (default 10 min on a Pi): 1. the primary service reports **healthy** in Docker; then 2. the `x-app.healthCheck` URL (else the instance's main URL) returns 2xx/3xx and, if given, contains the expected body string (Nextcloud `status.php`: `"maintenance":false` and `"needsDbUpgrade":false`).

A climbing container restart count (crash loop) fails the gate early instead of waiting out the timeout. Any failure: automatic rollback.

4.4 Minor vs major

- **Minor** (same major, `isMajorUpgrade` false): data format is compatible or the App migrates forward safely. Allowed from the Console and at dock time.
- **Major**: migrations may be one-way and slow. Allowed only if the new App declares the old major in `x-app.updatesFrom` (Nextcloud lists only its previous major, because Nextcloud can't skip majors), and only with a Borg backup made in the last hour, because rollback then means restoring the snapshot, not just the old images.

4.5 Relation to dock-time upgrade (#76), #74 and #77

76 has two halves: *detect* a newer version when a disk is docked, and *perform* the upgrade. The second is exactly `updateApp`. Build the command first; then the dock path only writes an "update available" hint, when the newer bundle is on a disk docked on any Engine in the shared store (Koen, 2026-10-28). If it is on another Engine, the update first fetches the bundle over the school LAN (§6, step 2). The Console offers the command. The Engine never updates on its own at dock time, because a teacher may dock a disk only to copy something.

The Upgrade Disk (#74) and Engine self-upgrade (#77) are about the Engine, not Apps. They can reuse the same pattern (snapshot, health gate, rollback, `Operation` record), but they're not part of this change.

5. Recommended option

In-place swap (A), with a local snapshot, a health gate and automatic rollback; minor versions from the Console and dock-time hints; major versions only when the App opts in and a fresh Borg backup exists.

Why: the instance identity (ID, name, port, backups) is what the school and the other Pis know, and A keeps it. B's double data and duplicate-name problems are exactly what #230 is trying to remove. The local snapshot gives B's "old one is untouched" safety at the cost of one data copy that's deleted after success. The health gate turns "it started" into "it works", which is what makes automatic rollback safe without IT staff.

6. Flow and failure rules

1. **Check** (no changes). Any failure ends with a refusal code and nothing changed: `NOT_DOCKED` (instance's disk not docked here), `BUSY` (operation or `ResourceLock` on instance or disk),

`SOURCE_MISSING` (no readable `apps/<newAppId>/` with its images), `DIFFERENT_APP` (other App name), `NOT_NEWER`, `MAJOR_NOT_ALLOWED` (not in `updatesFrom`, or no `--allow-major`), `NEEDS_BACKUP` (major without a Borg backup under 1 h old), `NO_SPACE` (§4.2).

2. **Fetch** (only when the source disk is docked on another Engine; nothing is stopped, the live instance keeps running):
3. *What travels:* only `apps/<newAppId>/` (`compose.yaml`, bundle files, `services/*.tar`). The instance's `.env`, `data/` and volumes never travel. Image tars whose image is already in the target's Docker may be skipped.
4. *Where:* into a pi-owned staging folder on the instance's App Disk, `.pre-update/incoming/`, written as pi. File modes are kept (Nextcloud's `docker-entrypoint-hooks.d` scripts must stay executable), so the copy is a tar- or rsync-style stream that keeps permissions, not per-file HTTP. Root-owned files travel as data; the existing install code sets ownership. No new sudoers or root helpers.
5. *Space:* checked before the copy starts: staged bundle + snapshot + about 10% margin; otherwise `NO_SPACE` naming the full disk.
6. *Integrity:* each bundle carries a build-time `SHA256SUMS` (Kid adds it to app-kolibri, app-nextcloud, app-kiwix, app-milkwise). Fetch verifies every file against it, not against a hash taken on the source disk.
7. *Resume:* a retry resumes partial files from their byte offset. A completed file failing its checksum is moved aside and fetched again. After an abort (`SOURCE_MISSING`, user cancel, or a second checksum failure on one file) the whole staging folder is moved aside (quarantine, never deleted) and the next attempt starts clean.
8. *Failures:* network drop or source disk undocked → `SOURCE_MISSING` or the new `TRANSFER_FAILED`; nothing on the instance has changed.
9. *Heat:* pause above 75 °C and resume below 65 °C on either Engine. That is a wait (operation state `fetchPaused`, reason `temperature`), not `TRANSFER_FAILED`.
10. *Network:* Engine to Engine over the school LAN only, never Tailscale. Transport is Axle's choice (§10).
11. **Journal and snapshot**: write the journal to `.pre-update/` on the disk; stop the instance and wait until its containers are really gone (fixes the stop/start race in #230); move the old bundle files there; copy `data/` and the named volumes (minus `snapshotExclude`).
12. **Swap**: copy the new bundle (from `.pre-update/incoming/` when fetched) into the instance and re-apply the Engine's compose adjustments with the **same code `installApp` uses** (`.env` kept: port, pass, hostname, ip; `x-app.version`, `instanceName`, files/USB mounts); load the new image tars; set `instanceOf` to the new App ID. Old images stay in Docker.
13. **Start and health gate** (§4.3).
14. **Success**: operation Completed; trace reports old → new. 24 h after the gate passed, the snapshot moves to quarantine (never deleted by the Engine). The old bundle `apps/<oldAppId>/` stays on the disk.

15. **Failure in 4 or 5**: stop; restore the old bundle **and the data copy** (`data/` and volumes) from the snapshot; restore `instanceOf` ; start the old version (its images are still in Docker); operation Failed with `ROLLED BACK: <why>` . If rollback itself fails, leave the instance Stopped with a `statusCondition` explaining it and the snapshot intact.
16. **Disk pulled or Engine restarted mid-update**: the journal on the disk is the source of truth; the store operation is only the visible record. Whichever Engine docks that disk next sees the unfinished journal and rolls back (step 7) **before anything on the disk starts**. Recovery strategy for `upgradeApp` stays `'fail'` .

7. Store and command shape

7.1 Command

```
updateApp <instanceIdOrName> <newAppId> [--source <diskId>] [--allow-major]
```

- Scope `engine` ; resolved with `resolveInstanceArg` / `resolveDiskArg` (IDs, never disk names). Without `--source` , it prefers a disk docked on this Engine that has `apps/<newAppId>/` , otherwise one docked on another Engine in the shared store (then Fetch runs).
- GitHub as a source (when online) is a dev/ops command option only, for tagged releases; the Console never offers it.
- New Engine capability `updateApp` , so the Console knows the command exists.

7.2 Store

- Operation: `kind: 'upgradeApp'` , subject = instance, args `{fromVersion, toVersion, major, estimatedMinutes, healthTimeoutMs, sourceDiskId}` , steps Check / Fetch / Snapshot / Swap / Start / Health, plus a state `fetchPaused` with reason `temperature` , args also `sourceEngineId` , with the existing `currentStep` / `stepLabel` on the instance.
- Instance: `instanceOf` changes only on success; new optional `previousAppId?: AppID | null` ("Updated from 0.15.5").
- `instance.updateAvailable?: { appId, sourceEngineId, sourceDiskId, fromVersion, toVersion, major, estimatedMinutes } | null` . Set when a newer bundle (per `updatesFrom`) is on any disk docked on any Engine known in the shared store; cleared when that disk undocks anywhere, on Engine boot, and when the update finishes. `estimatedMinutes` = transfer estimate (0 on the same Engine) + snapshot copy estimate + health timeout.
- App bundle (`x-app`): optional `healthCheck: {path, expect?}` , `healthTimeout` , `updatesFrom: [<major> … ]` , `migrationNote` , `snapshotExclude: [<path> … ]` .

7.3 Console (Pixel)

- Badge **Update available** from `updateAvailable`, only if the Engine has the `updateApp` capability.
- When the source is on another Engine, badge and confirm say where it comes from ("from the disk in ") and that copying it over first takes time; the live App keeps working during the copy.
- **Not now** hides the badge on that browser only; nothing is written to the store.
- Confirm: "Update Kolibri from 0.15.5 to 0.15.6? Your data is kept. A copy is made first. This takes about N minutes, and Kolibri is unavailable meanwhile." For a major, the App's `migrationNote` is shown, and **Update** is disabled with "Make a backup first" when there is no Borg backup under 1 h old.
- Paused fetch: "Paused while cools down. The copy continues by itself."
- Progress: the step label (Checking, Copying from , Making a copy, Updating, Starting, Checking it works).
- Result: "Kolibri is updated to 0.15.6", or "The update didn't work, so Kolibri is back on 0.15.5 with all its data. Reason: …". Refusal codes map to plain sentences.

8. Test plan

- **Unit**: argument parsing; every refusal code; minor/major rule; free-space measurement incl. named volumes and `snapshotExclude`; health gate (Docker healthy, body string, crash-loop early fail); `.env` and Engine-adjustment merge; rollback on every failing step (mocked docker).
- **Engine integration (box, test mode)**: sample App 1.0 → 1.1 with data kept; 1.1 with a broken health URL rolls back to 1.0 with the same data; refusal when busy, when the disk is missing, for a major version without opt-in.
- **Pi (pool Pi, claimed)**: real Kolibri and Nextcloud (DB volume) updates with real data; pull the disk during Swap and re-dock on **another** Engine, check it rolls back before starting; reboot during Start, check rollback on boot; Engine restarted during the health gate; `updateAvailable` cleared on undock and boot.
- **Two Engines (pool Pis, claimed)**: source disk on Engine A, instance on Engine B; full update via Fetch. Network loss during Fetch (abort, nothing changed; retry resumes); source disk pulled during Fetch (`SOURCE_MISSING`, staging quarantined); corrupted file (refetched, second failure aborts); thermal pause and resume. Atlas measures a real multi-GB copy between pool Pis (Wi-Fi and wired) for the transfer estimate.
- **Console**: "Update available" badge, confirm, progress and the rolled-back message (mock store + Playwright).
- **App (Kid)**: harness test "update from previous release with real data" for app-kolibri, app-nextcloud, app-kiwix, app-milkwise; measured first-start times after migration set each App's `healthTimeout`.

9. Design review

- **Console (Pixel):** badge gated on the capability, local-only "Not now", confirm/progress/result wording, migration note and the backup gate for majors (§7.3).
- **Engine (Axle):** named volumes in the snapshot and the space check, the on-disk journal with rollback by whichever Engine docks next, `updateAvailable` scoping and fields, Docker-healthy plus body-string health gate with crash-loop early fail, rollback of data (not just images), reuse of the install compose code, refusal codes and the `updateApp` capability.
- **App Disks (Kid):** Nextcloud `updatesFrom` limited to the previous major, `snapshotExclude` for Kolibri `content/`, the `x-app` fields in four App repos, the harness update test and measured timings.
- **Ops (Atlas):**
- *Space:* the free-space check covers both disks: the App Disk (copy of `data/` and the named volumes minus `x-app.snapshotExclude`, plus the new image tars, plus about 10% margin) and the system disk's `/var/lib/docker`, because old and new images sit side by side. No hardlink (`cp -al`) snapshots, since SQLite and MariaDB change files in place. Old images are cleaned up only when the snapshot is deleted (moved to quarantine), not before.
- *Backups:* the `NEEDS_BACKUP` refusal text says "Dock a Backup Disk and make a backup first". In the field this effectively means no major updates without a Backup Disk, which is accepted.
- *Reboot safety:* add an integration test "Engine restarted during the health gate" next to the disk-pull test (pool Pis and the `idea` Pi reboot daily at 05:00). The Engine logs the CPU temperature during the swap. Fleet runs keep the 75 °C pause rule.
- *Rollout:* no new sudoers or root helpers (`docker load` and compose already run as pi). Order: Engine PR tested on claimed pool Pi idea04, then merge, then deploy to idea01, then golden. Kits only in an overnight window Koen opens.
- *Open questions:* Atlas's answers agree with the decisions below (per-App `healthTimeout` default 10 min; Borg backup required for majors; refuse without a copy; GitHub update is ops/dev-only; one instance at a time, later Pi by Pi stopping at the first rollback; keep the old bundle until the snapshot is deleted).

Decided in review (former open questions): health timeout per App with a 10 min default; a major update needs a Borg backup under 1 h old; no space means refuse (`NO_SPACE`); GitHub updates only as a dev/ops command for tagged releases; one instance at a time; the old bundle stays on the disk.

- **Fetch (added after Koen's 2026-10-28 decision):**
- *Kid:* copy only `apps/<newAppId>/` ; `.env` , data and volumes never travel; keep file modes (stream, not per-file HTTP); verify against a build-time `SHA256SUMS` in each bundle, which Kid adds to the four App repos; optionally skip image tars already in the target's Docker.

- *Atlas:* plain HTTP or websocket stream inside the LAN with checksums for integrity, no SSH or TLS (the Pi 4 has no hardware AES); per-file byte-offset resume; 75/65 °C pause on either Engine as a wait; staging written as pi, no new sudoers; free-space check (bundle + snapshot + margin) before the copy, `NO_SPACE` naming the disk; Atlas will measure a real copy.
- *Pixel:* separate state `fetchPaused` (reason `temperature`) and the "Paused while cools down" text.
- *Lead (Steve):* resume from byte offset; per-file refetch on checksum failure; staging quarantined (never deleted) after an abort, next attempt starts clean.
- *Axle:* pending (transport, §10).

10. Open questions

Two items remain:

1. **Measured timings (Kid):** Kolibri 0.15.5 to main with real data and Nextcloud one major step, measured on idea04: first-start times after migration, to set each App's `healthTimeout` and the `estimatedMinutes` copy rate.
2. **Fetch transport (Axle):** the Engine already copies Apps between Engines: `copyApp` cross-engine (`src/data/CopyMoveApp.ts`, Engine main `2583608`) pushes from the **source** Engine with `rsyncToPeer` (`src/utils/rsync.ts`) over ssh with per-Pi Engine keys (`src/utils/peerSsh.ts`), to the peer's LAN address from `network.connections` (`getEngineAddress`), where the peer's gate runs `idea-app-data receive-app` / `receive-service`. That keeps file modes and resumes like rsync and needs no new sudoers, but it is SSH (Atlas prefers no SSH/TLS for speed on the Pi 4) and it is source-initiated, so the target would ask the source Engine via `sendCommand` to push into its staging folder. Choose: reuse that path (staging under `.pre-update/incoming/` may need a new gate verb or reuse `receive-app` into `apps/<newAppId>/`), or a new plain LAN HTTP/websocket stream (tar with modes, byte-offset resume) per Atlas. Either way: LAN only, never Tailscale; no new root helpers.